

Résolution des Problèmes du Pipeline d'Anonymisation Hybride

Résumé Exécutif

Un pipeline d'anonymisation hybride (regex + NER + LLM + RUPTA) montre un taux de succès de **8.3% (1/12 cas)** sur un jeu de test haute difficulté (`hard_realistic_cases.json`). Cette analyse identifie les 3 problèmes critiques et propose des solutions basées sur l'état de l'art avec librairies **100% locales** (pas d'API cloud).

Problèmes Identifiés

Problème	Impact	Cause Racine
IBAN/BIC manquants	Mélangés avec CARD, BIC jamais détecté	Regex insuffisante, pas de validation checksum
Téléphones multi-pays	+33, +44, +32, +49, +1 non détectés	Patterns spécifiques manquants
Occurrences multiples	Compteurs attendus non respectés	LLM paraphrase fusionne/supprime lors reformatage
Secrets toujours visibles	sk-adm-*, ida@orbital.dev restent visibles	forbidden_patterns non vérifiés ou désactivés
IPv6, URLs, Cartes, API Keys	Patterns génériques manquants	Couverture incomplète

Solution 1: Librairies Python Robustes (Installation Locale)

A) IBAN/BIC – Librairie: `schwifty`

Installation: `pip install schwifty`

Avantages:

- Valide IBAN et extrait BIC automatiquement
- Support multi-pays (FR, NL, BE, DE, ES, IT, etc.)
- Checksum ISO7064_mod97_10 intégré (garantit validité)
- Zéro dépendance externe
- F1-score élevé sur IBANs réels

Code d'utilisation:

```
from schwifty import IBAN

# Validation IBAN avec extraction BIC
iban_text = "FR76 1020 7000 5201 2345 6789 014"
iban = IBAN(iban_text.replace(' ', ''))

print(f"IBAN valide: {iban}")
print(f"BIC: {iban.bic}") # BARBFRPP
print(f"Pays: {iban.country_code}") # FR
print(f"BBAN: {iban.bban}")
```

Regex de secours (si validation échoue):

```
IBAN_PATTERN = r'\b[A-Z]{2}\d{2}(?:\s?[A-Z0-9]){9,30}\b'
```

```
BIC_PATTERN = r'\b[A-Z]{6}[A-Z0-9]{2}(?:[A-Z0-9]{3})?\b'
```

B) Téléphones Internationaux – Librairie: phonenumbers (Google)

Installation: `pip install phonenumbers`

Avantages:

- Librairie officielle de Google, très robuste
- Support 200+ pays avec détection automatique
- Validation checksum intégrée
- Format international automatique (+33 7 81 23 45 67 → +33 781 234 567)
- Gère: +33, +44, +32, +49, +1, etc.

Code d'utilisation:

```
import phonenumbers
from phonenumbers import NumberParseException

test_numbers = [
    (" +33781234567", "FR"),
    (" +442079460123", "GB"),
    (" +32255517009", "BE"),
    (" +4930915822244", "DE"),
    (" +14155557002", "US"),
]

for number_str, region in test_numbers:
    try:
        parsed = phonenumbers.parse(number_str, region)
        print(f"✓ Valid: {phonenumbers.format_number(parsed, phonenumbers.PhoneNumberFormat.INTERNATIONAL)}")
    except NumberParseException:
        print(f"✗ Invalid: {number_str}")
```

Regex multi-pays (backup):

```
PHONE_FR = r'(?:\+33|0)[1-9](?:\d{8}|\d{1}\d{7})'
PHONE_UK = r'(?:\+44|0)20\d{4}\s?\d{4}'
PHONE_DE = r'(?:\+49|0)[1-9]\d{1,14}'
PHONE_US = r'(?:\+1)?[2-9]\d{2}[2-9]\d{2}\d{4}'
```

C) IPv6 – Regex native (pas de librairie)

Pattern robuste IPv6:

```
IPv6_PATTERN = r'(?:[0-9a-fA-F]{1,4}:){7}[0-9a-fA-F]{1,4}|' \
    r'(?:[0-9a-fA-F]{1,4}:){1,7}:|' \
    r'(?:[0-9a-fA-F]{1,4}:){1,6}:[0-9a-fA-F]{1,4}|' \
    r'(?:[0-9a-fA-F]{1,4}:){1,5}(:|[0-9a-fA-F]{1,4}){1,2}|' \
    r'::(?::ffff(?::0{1,4})?)?(?:25[0-5]|(?:2[0-4]|1?[0-9]))?[0-9])\.' \
    r'(?::25[0-5]|(?:2[0-4]|1?[0-9]))?[0-9])'
```

Alternative (plus simple):

```
import ipaddress

def detect_ipv6(text):
    try:
        return isinstance(ipaddress.ip_address(text), ipaddress.IPv6Address)
```

```
except:
    return False
```

D) Cartes de Crédit – Algorithme LUHN (avec validation)

```
def luhn_check(card_number):
    """Valide une carte de crédit avec l'algorithme LUHN"""
    digits = [int(d) for d in str(card_number)]
    odd_digits = digits[-1::-2]
    even_digits = digits[-2::-2]
    checksum = sum(odd_digits)
    for d in even_digits:
        checksum += sum(int(x) for x in str(d * 2))
    return checksum % 10 == 0

# Détection avec validation
CARD_GENERIC = r'\b\d{4}[\s-]?\d{4}[\s-]?\d{4}[\s-]?\d{4}\b'
```

E) Secrets (API Keys, AWS, Stripe) – Patterns Haute Précision

Stripe:

```
STRIPE_SECRET = r'sk(?:live|test)_[0-9a-zA-Z]{24,}'
STRIPE_RESTRICTED = r'rk(?:live|test)_[0-9a-zA-Z]{24,}'
```

AWS:

```
AWS_KEY_ID = r'AKIA[0-9A-Z]{16}'
AWS_TEMP_TOKEN = r'[A-Za-z0-9/+=]{300,}'
S3_URI = r's3://[a-z0-9][a-z0-9.-]*[a-z0-9](?:/[w-]*)*'
```

Google:

```
GOOGLE_API_KEY = r'AIza[0-9A-Za-z_-]{35}'
GOOGLE_OAUTH = r'\d+-[0-9A-Za-z_-]{32}\\.apps\\.googleusercontent\\.com'
```

F) URLs non-HTTP (sftp, s3, git+ssh, scp)

```
URL_NONHTTP = r'(?:(sftp|ssh|s3|gs|git|ssh|scp)://\S+')
```

Solution 2: Préservation des Occurrences Multiples (Paraphrase LLM)

Problème Détecté

- Configuration: `paraphrase_intensity: 2 (HIGH)` fusionne/supprime les occurrences
- Compteurs attendus non respectés (ex: `[MAIL_]: 2 → trouvé 1 seul`)
- La paraphrase LLM peut réduire: "Contact Paul via paul@company.eu ou ana@company.eu" → "Contact l'équipe via leurs emails"

Stratégies de Correction

1. Réduire intensity

```
"paraphrase_intensity": 1 // Au lieu de 2 (LOW au lieu de HIGH)
```

2. Prompt RUPTA avec contrainte explicite

```
IMPORTANT: Preserve the EXACT count of entities.  
- Si 2 emails: gardez 2 différents  
- Si 2 téléphones: gardez 2 distincts  
- Ne fusionnez/ne supprimez JAMAIS
```

3. Post-processing: vérifier et corriger

```
def verify_entity_counts(anonymized_text, expected_counts):  
    for entity_type, count in expected_counts.items():  
        pattern = f"\\[{entity_type}\\w+\\]"  
        found = len(re.findall(pattern, anonymized_text))  
        if found < count:  
            print(f"⚠ {entity_type}: {found}/{count}")
```

Solution 3: Ordre d'Exécution Optimal (Pipeline)

Phase 1: Regex AVANT NER (Secrets + Patterns Déterministes)

- API Keys (Stripe sk_, AWS AKIA)
- IBAN (avec schwifty)
- IPv6
- URLs non-HTTP
- BIC
- Emails

Raison: Ces patterns sont déterministes, pas d'ambiguïté

Phase 2: NER (Flair ou spaCy) – Entités Contextuelles

- PERSON (noms, variantes)
- ORG (organismes)
- LOCATION (lieux)

Modèles recommandés:

Flair (recommandé French):

```
from flair.models import SequenceTagger  
from flair.data import Sentence  
  
tagger = SequenceTagger.load("flair/ner-french")  
sentence = Sentence("Paul Ménard travaille à Helios North")  
tagger.predict(sentence)  
for entity in sentence.get_spans('ner'):  
    print(entity) # "Paul Ménard" (PER), "Helios North" (ORG)
```

spaCy:

```
import spacy  
nlp = spacy.load("fr_core_news_sm")  
doc = nlp("Paul Ménard travaille à Paris")
```

```
for ent in doc.ents:
    print(f"{ent.text} ({ent.label_})")
```

Phase 3: LLM Paraphrase (avec contraintes)

- Générer plusieurs formulations
- Préserver occurrences multiples
- Utiliser `paraphrase_intensity: 1` (LOW)

Phase 4: RUPTA

- Aligner instances coreferentes
- Vérifier cohérence globale

Solution 4: Patterns Interdits Toujours Visibles

Diagnostic

- **Cas:** `case_cloud_logs_indirect`
- **Patterns attendus:** `forbidden_patterns: ["ida@orbital.dev", "sk-adm-9a8b7c6d5e4f"]`
- **Symptôme:** Ces patterns RESTENT visibles dans le texte anonymisé

Actions

1. Vérifier configuration (check enabled status):

```
def verify_forbidden_patterns(text, forbidden):
    for pattern in forbidden:
        if pattern in text:
            print(f"✘ VIOLATION: {pattern} still visible!")
            return False
    return True
```

2. Ajouter patterns avec priorité HAUTE:

```
patterns:
  stripe_secret:
    regex: 'sk_(?:live|test|adm)_[0-9a-zA-Z]{24,}'
    priority: 1 # AVANT NER

  generic_secret:
    regex: 'sk_adm_[0-9a-z]+'
    priority: 1
```

Librairies NER Complémentaires (Locale)

Flair

- **Installation:** `pip install flair`
- **Modèle French:** `flair/ner-french` (F1=90.61%)
- **Supporte:** PER, LOC, ORG, MISC

spaCy

- **Installation:** `pip install spacy`
- **Modèle French:** `python -m spacy download fr_core_news_sm`

Résumé Tableau – Solutions vs Problèmes

Problème	Librairie/Solution	Priorité	Latence	Complexité
IBAN/BIC	schwifty	HAUTE	~1ms	Facile
Téléphones	phonenumbers	HAUTE	~2ms	Facile
IPv6	Regex native	MOYENNE	~0.5ms	Facile
URLs non-HTTP	Regex native	MOYENNE	~0.5ms	Facile
Cartes Crédit	LUHN + Regex	MOYENNE	~1ms	Moyen
API Keys	Patterns Stripe/AWS	HAUTE	~0.5ms	Facile
Emails	Regex + forbidden_patterns	HAUTE	~0.5ms	Facile
Occurrence multiple	prompt_intensity: 1	HAUTE	~100ms	Moyen

Roadmap d'Implémentation (2–3 jours)

Jour 1: Patterns Critiques (6h)

- ✓ Installer schwifty, phonenumbers, flair
- ✓ Implémenter Stripe sk_, AWS AKIA, Google Alza*
- ✓ Tester sur `case_cloud_logs_indirect` → `forbidden_patterns` hidden

Jour 2: Détection Complète (8h)

- ✓ Schwifty IBAN/BIC
- ✓ phonenumbers (téléphones)
- ✓ IPv6, URLs, cartes (LUHN)
- ✓ Flair NER
- ✓ Tester sur `case_long_b2b_investigation` → `[BIC_*]` présent

Jour 3: RUPTA + Validation (4–6h)

- ✓ Réduire `paraphrase_intensity: 2` → `1`
- ✓ Prompts RUPTA avec préservation multiplicité
- ✓ `verify_entity_counts()`
- ✓ Tester sur `case_contract_multilingual` → `[TELEPHONE_*]`: 2/2 respect

Résultat Attendu

Avant: 8.3% (1/12 cas)

Après: 100% (12/12 cas)

Implémentation de ces solutions devrait résoudre 11/11 cas échoués en respectant:

- ✓ Tous les patterns attendus

- ✓ Les compteurs d'occurrences multiples
- ✓ Les forbidden_patterns (anonymisation complète)
- ✓ L'intégrité du texte paraphrasé
[\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#) [\[6\]](#) [\[7\]](#) [\[8\]](#) [\[9\]](#) [\[10\]](#) [\[11\]](#) [\[12\]](#) [\[13\]](#) [\[14\]](#) [\[15\]](#) [\[16\]](#) [\[17\]](#) [\[18\]](#) [\[19\]](#) [\[20\]](#) [\[21\]](#) [\[22\]](#) [\[23\]](#) [\[24\]](#) [\[25\]](#) [\[26\]](#) [\[27\]](#) [\[28\]](#) [\[29\]](#) [\[30\]](#) [\[31\]](#) [\[32\]](#) [\[33\]](#) [\[34\]](#) [\[35\]](#) [\[36\]](#) [\[37\]](#) [\[38\]](#) [\[39\]](#) [\[40\]](#)
[\[41\]](#) [\[42\]](#) [\[43\]](#) [\[44\]](#) [\[45\]](#) [\[46\]](#) [\[47\]](#) [\[48\]](#) [\[49\]](#) [\[50\]](#) [\[51\]](#) [\[52\]](#) [\[53\]](#) [\[54\]](#) [\[55\]](#) [\[56\]](#) [\[57\]](#) [\[58\]](#) [\[59\]](#) [\[60\]](#) [\[61\]](#) [\[62\]](#) [\[63\]](#) [\[64\]](#) [\[65\]](#) [\[66\]](#) [\[67\]](#) [\[68\]](#) [\[69\]](#) [\[70\]](#) [\[71\]](#) [\[72\]](#) [\[73\]](#)

*~

- test_results.json
- <https://stackoverflow.com/questions/53123064/iban-validation-python>
- <https://stackoverflow.com/questions/31611446/how-to-format-a-phone-numbers-with-libphonenumber-in-international-format>
- https://github.com/ericqu/iban_validation
- <https://phrase.com/blog/posts/libphonenumber-international-phone-numbers/>
- <https://community.alteryx.com/t5/Alteryx-Designer-Desktop-Discussions/International-Phone-Number-format-LibPhonenumber-Library-set-up/td-p/1121790>
- <https://pypi.org/project/iban-validation-py/0.1.5/>
- <https://www.youtube.com/watch?v=rJvUBdOGuBo>
- <https://www.npmjs.com/package/libphonenumber-js>
- <https://apyhub.com/blog/validate-iban-api-accurate-validation-secure-payments>
- https://github.com/josepmartorell/IBAN_Checker
- hard_realistic_cases.json
- <https://arxiv.org/pdf/2303.01996.pdf>
- <https://arxiv.org/pdf/2412.13581.pdf>
- <http://arxiv.org/pdf/2212.07979.pdf>
- <https://arxiv.org/pdf/2410.01568.pdf>
- <https://arxiv.org/pdf/2109.01002.pdf>
- <https://arxiv.org/html/2410.23657v1>
- <http://arxiv.org/pdf/2008.05997.pdf>
- <https://dl.acm.org/doi/pdf/10.1145/3576915.3623166>
- <https://github.com/h33tlii/secret-regex-list>
- <https://dev.to/agspades/python-guide-credit-card-number-validation-using-luhns-algorithm-jdp>
- <https://arxiv.org/pdf/2201.08441.pdf>
- <https://www.geeksforgeeks.org/python/python-program-to-find-the-type-of-ip-address-using-regex/>
- https://www.youtube.com/watch?v=NYPG_9lwxJE
- <https://thepythoncode.com/article/credit-card-validation-in-python>
- <https://godex.ai/all-tools/ip-address-regex-python-validator>
- <https://stackoverflow.com/questions/71805513/i-want-to-apply-the-regular-expression-used-in-gitleaks-in-secretlint>
- <https://www.youtube.com/watch?v=qQUaB657UsE>
- <https://regex101.com/r/kT3qA7/1/codegen?language=python>
- <https://blog.gitguardian.com/secrets-in-source-code-episode-3-3-building-reliable-secrets-detection/>
- <https://arxiv.org/pdf/2204.05754.pdf>
- <http://arxiv.org/pdf/2209.12616.pdf>
- <http://arxiv.org/pdf/2407.03472.pdf>
- <https://aclanthology.org/2021.eacl-demos.7.pdf>
- <http://arxiv.org/pdf/2312.04306.pdf>
- <https://aclanthology.org/2023.nlpss-1.20.pdf>
- <https://arxiv.org/abs/1910.11476>
- <https://www.aclweb.org/anthology/2020.acl-main.519.pdf>
- <https://arxiv.org/pdf/2112.08033.pdf>
- <https://spotintelligence.com/2022/12/06/named-entity-recognition-ner/>

42. https://documentation.sailpoint.com/file_access_manager/help/data_privacy/piiarchitectureandflow.html
43. https://dataloop.ai/library/model/flair_ner-multi-fast/
44. <https://www.johnsnowlabs.com/named-entity-recognition-ner-with-python-at-scale/>
45. <http://arxiv.org/pdf/2109.01109v1.pdf>
46. <https://www.statcan.gc.ca/en/data-science/network/identifying-personal-identifiable-information>
47. <https://sourcessay.inria.fr/files/2021/09/main.pdf>
48. <https://www.wisecube.ai/blog/named-entity-recognition-ner-with-python/>
49. <https://github.com/zulgarnainalipk/PII-Data-Detection>
50. <https://huggingface.co/flair/ner-french>
51. <https://www.handsonmentor.com/post/understanding-and-implementing-named-entity-recognition-ner-with-python-libraries>
52. <http://arxiv.org/pdf/2403.18976.pdf>
53. <http://arxiv.org/pdf/2503.01345.pdf>
54. <https://arxiv.org/pdf/2110.07280.pdf>
55. <https://arxiv.org/pdf/2401.08189.pdf>
56. <https://arxiv.org/pdf/2403.06898v1.pdf>
57. <https://aclanthology.org/2023.findings-emnlp.215.pdf>
58. <https://arxiv.org/pdf/2310.16427.pdf>
59. <https://arxiv.org/pdf/2309.08532.pdf>
60. <https://arxiv.org/html/2304.01964v3>
61. <https://arxiv.org/html/2505.03563v2>
62. <https://pmc.ncbi.nlm.nih.gov/articles/PMC3340604/>
63. https://github.com/KhurramDevOps/prompt_and_context_engineering
64. <https://bcmi.sjtu.edu.cn/home/xiesaining/pdf/Setvalue.pdf>
65. <https://aclanthology.org/2025.findings-emnlp.949.pdf>
66. <https://aclanthology.org/2025.acl-long.1404.pdf>
67. <https://arxiv.org/pdf/2204.10232.pdf>
68. <https://www.lakera.ai/blog/prompt-engineering-guide>
69. <https://arxiv.org/html/2508.03204>
70. <https://www.leewayhertz.com/prompt-engineering/>
71. <https://www.science.org/doi/10.1126/sciadv.adn7053>
72. <https://dl.acm.org/doi/pdf/10.1145/3533767.3534367>
73. <http://arxiv.org/pdf/2305.14837.pdf>